



**DESARROLLO DE SOFTWARE PARA MÓVILES
DSM 941 G01T**

Trabajo de investigación

Docente:

Alexander Alberto Siguenza Campos

Estudiantes

Nombres	Apellidos	Carnet
Carlos David	Guevara Martínez	GM172474
Fernando Josue	Torres Lara	TL222729
Cristian Omar	Pineda Campos	PC222718
Geovany Arturo	Pineda Fuentes	PF211251

Grupo 9

Desarrollo de una Aplicación de Control de Gastos Personales con Firebase y Kotlin en Android Studio.

Enlace de video

<https://drive.google.com/drive/folders/1BjE96V-cY2yOIUT9DQkYSiGANwIekXdz>

Enlace de repositorio

https://github.com/MrUnknown023/FORO2_DSM

Índice

Introducción.....	4
Investigación sobre Autenticación en Firebase.....	5
Tipos de Autenticación	5
Casos de Uso	6
Desarrollo Tecnico.....	7
Configuración en Firebase Console.....	7
Integración del SDK en Android Studio.....	7
Implementación del Flujo de Login.....	7
Validaciones y Manejo de Errores	8
Capturas del proyecto en ejecución.	9
Conclusiones	16

Introducción

La seguridad es un pilar fundamental en cualquier aplicación móvil que gestione datos personales. En este proyecto, se ha implementado un sistema de autenticación robusto utilizando Firebase Authentication. Este servicio permite delegar la gestión de identidades a una plataforma escalable y segura, permitiendo a los usuarios acceder a sus datos de gastos de forma privada y consistente a través de diferentes dispositivos.

Investigación sobre Autenticación en Firebase

Tipos de Autenticación

Firebase ofrece múltiples métodos para verificar la identidad de los usuarios:

- Correo Electrónico y Contraseña: El método más tradicional. Permite crear una cuenta directamente en el sistema de Firebase.
- Proveedores de Identidad Federada (Google, Facebook, Apple, etc.): Permite a los usuarios iniciar sesión con sus cuentas existentes de otras plataformas, mejorando la tasa de conversión al no requerir nuevos formularios.
- Número de Teléfono: Utiliza el envío de un código SMS para verificar la posesión del dispositivo.
- Autenticación Anónima: Permite crear cuentas temporales que pueden ser convertidas en cuentas permanentes posteriormente.
- Custom Auth: Permite integrar Firebase con un sistema de autenticación propio ya

Característica	Ventajas	Limitaciones
Seguridad	Cifrado de nivel bancario, manejo automático de sesiones y tokens JWT.	Dependencia total de los servicios de Google.

Facilidad	SDKs listos para usar que reducen meses de trabajo a días.	Personalización limitada en flujos pre-construidos (FirebaseUI).
Costo	Generoso plan gratuito (Spark) para la mayoría de los métodos.	El envío de SMS (Phone Auth) tiene costos asociados tras superar el límite.
Integración	Sincronización nativa con Firestore, Realtime DB y Cloud Storage.	Requiere conexión a internet constante para la validación inicial.

Casos de Uso

- Correo/Contraseña: Aplicaciones corporativas o de nicho donde se requiere un registro específico.
- Google Sign-In: Aplicaciones de consumo general donde la rapidez de acceso es crítica.
- Phone Auth: Aplicaciones de mensajería o servicios de entrega (delivery) donde el número de teléfono es el identificador principal.

Desarrollo Tecnico

Configuración en Firebase Console

1. Creación del Proyecto: Se creó el proyecto "FORO2_DSM" en la consola de Firebase.
2. Registro de la App: Se vinculó el paquete de la aplicación `com.dsm.foro.controldegastos`.
3. Habilitación de Métodos: En la sección Authentication, se habilitaron los proveedores:
 - a. Correo electrónico/contraseña.
 - b. Google.
4. Huellas Digitales (SHA-1): Se registró la huella SHA-1 obtenida mediante el comando `signingReport` de Gradle para permitir el flujo de Google Sign-In.

Integración del SDK en Android Studio

Se agregaron las dependencias necesarias en el archivo `build.gradle.kts` (app):

- `com.google.firebase:firebase-auth-ktx`: Para la lógica de autenticación.
- `com.google.android.gms:play-services-auth`: Para la integración específica con Google.
- Además, se incluyó el archivo `google-services.json` en el directorio `app/` para configurar los IDs de cliente y claves de API.

Implementación del Flujo de Login

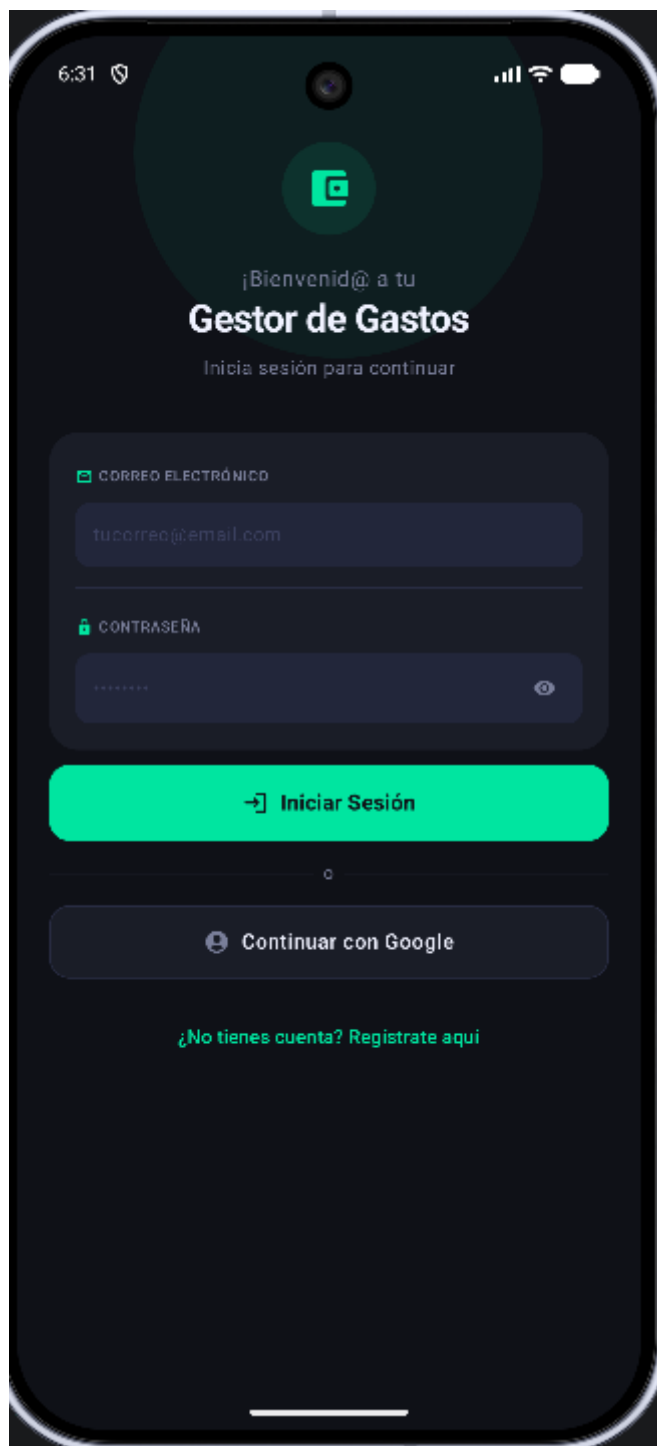
El desarrollo se estructuró siguiendo el patrón MVVM:

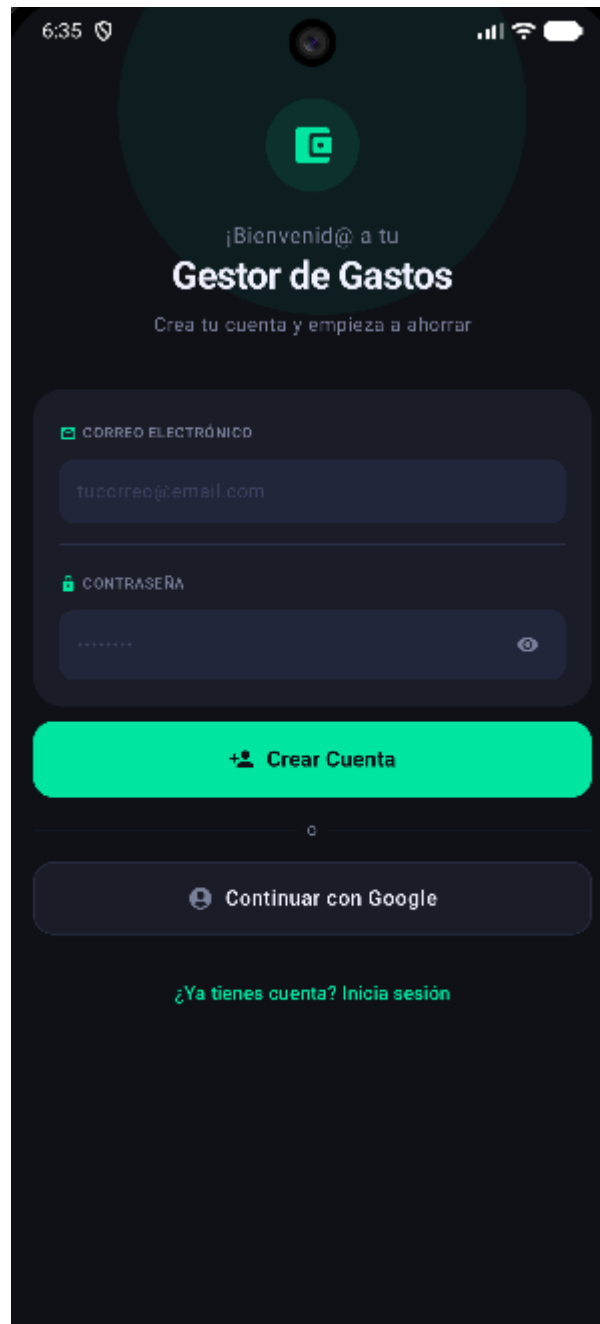
1. Repository (AuthRepository.kt): Gestiona las llamadas directas al SDK de Firebase (signInWithEmailAndPassword, signInWithCredential).
2. ViewModel (AuthViewModel.kt): Mantiene los estados de la interfaz (loading, error, sesión activa) y comunica la UI con el repositorio.
3. UI (LoginScreen.kt): Utiliza Jetpack Compose para mostrar los formularios. Implementa un ActivityResultLauncher para manejar la respuesta del selector de cuentas de Google.

Validaciones y Manejo de Errores

- Validación de Formulario: El sistema verifica que los campos no estén vacíos y que la contraseña tenga al menos 6 caracteres antes de enviar la solicitud.
- Control de Excepciones: Se implementaron bloques try-catch para capturar ApiException. Por ejemplo, se maneja el "Error 10" (desajuste de configuración) mostrando un mensaje amigable al usuario en lugar de permitir que la aplicación se cierre inesperadamente.

Capturas del proyecto en ejecución.





6:35

¡Bienvenid@ a tu

Gestor de Gastos

Crea tu cuenta y empieza a ahorrar

 CORREO ELECTRÓNICO

 CONTRASEÑA



 Crear Cuenta

 Continuar con Google

[¿Ya tienes cuenta? Inicia sesión](#)

6:33

Nuevo Gasto

NOMBRE

Ej. Almuerzo, Gasolina...

\$ MONTO

\$ 0.00

CATEGORÍA

Alimentación Transporte Salud

Educación Entretenimiento Otros

FECHA

Seleccionar fecha >

Guardar Gasto

Ver Historial

6:33

Nuevo Gasto

NOMBRE

Gasto hormiga

MONTO

\$ 1000

CATEGORÍA

Alimentación

Transporte

Salud

Educación

Entreténimiento

Otros

FECHA

15/05/2026

Ver Historial

6:34

Nuevo Gasto

NOMBRE

Ej. Almuerzo, Gasolina...

\$ MONTO

\$ 0.00

CATEGORÍA

Alimentación Transporte Salud

Educación Entretenimiento Otros

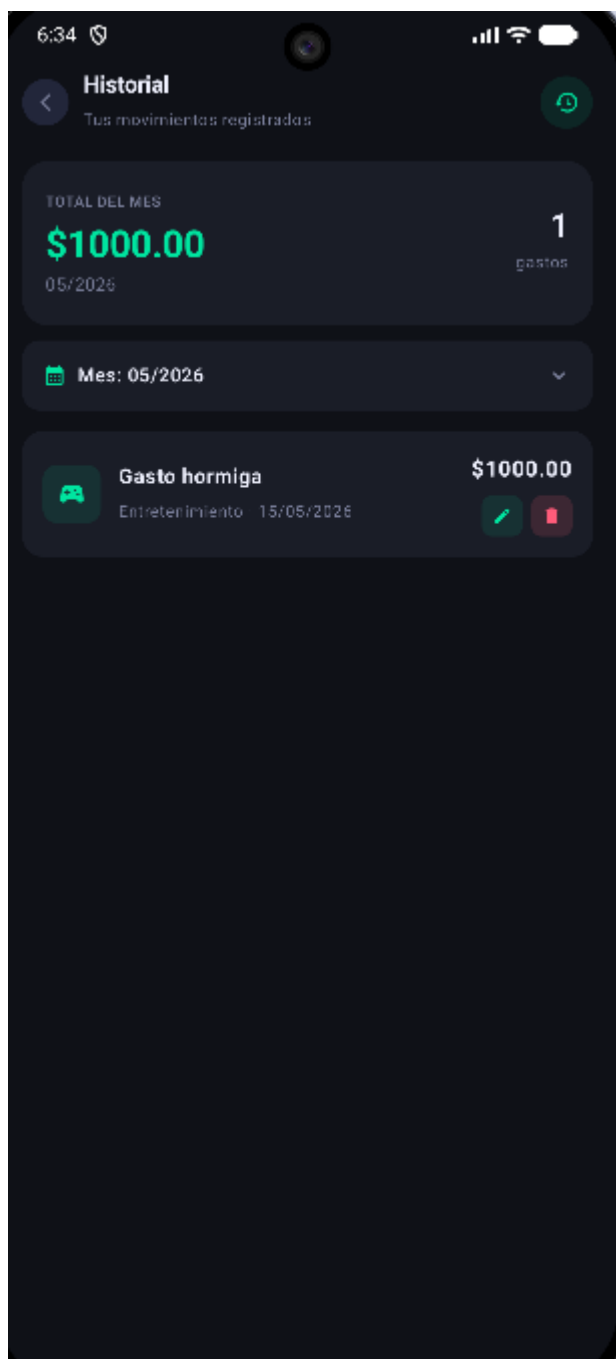
FECHA

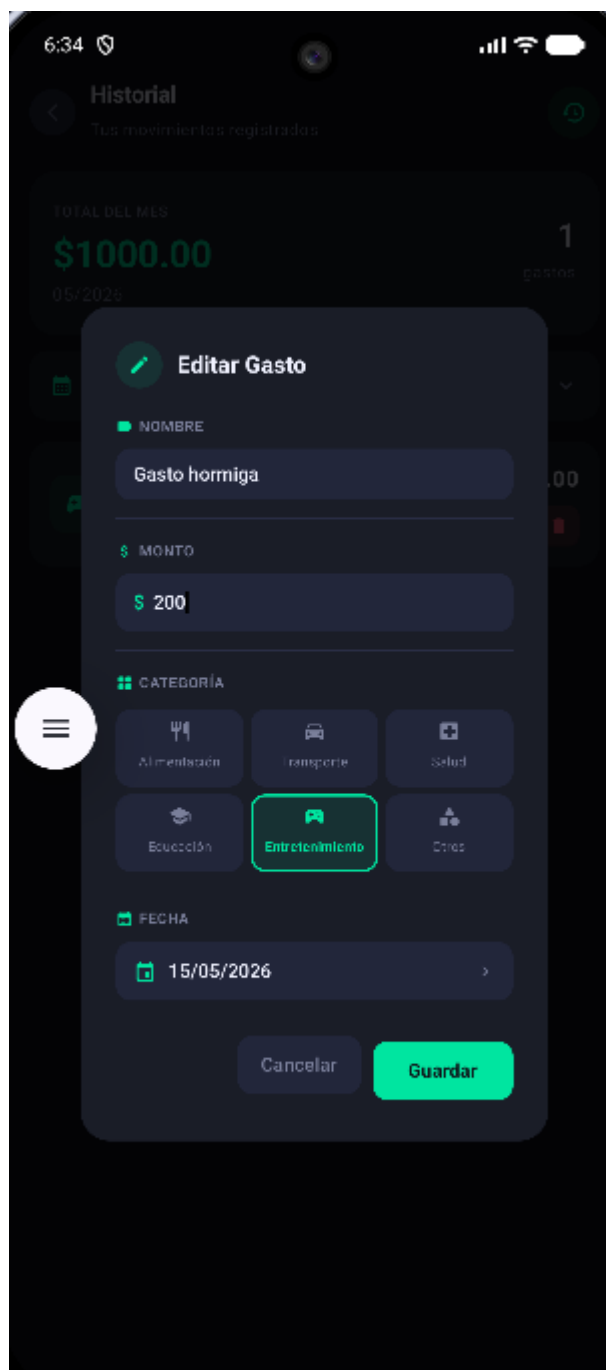
Seleccionar fecha >

¡Gasto guardado exitosamente!

Guardar Gasto

Ver Historial





Conclusiones

La implementación de Firebase Authentication en el proyecto ha demostrado ser una solución eficiente para delegar la complejidad de la seguridad. La facilidad para integrar múltiples proveedores (como Google) y la integración nativa con otros servicios como Firestore facilitan el desarrollo ágil de aplicaciones móviles profesionales. Se destaca la importancia de una correcta configuración en la consola (SHA-1 y Client IDs) como el punto más crítico para el éxito del despliegue.

Sobre Jetpack Compose: Considero que Jetpack Compose representa una mejora importante en la forma de desarrollar interfaces en Android. Durante el desarrollo de las pantallas del proyecto, se notó que trabajar con composables reutilizables permitió mantener un diseño más ordenado, consistente y fácil de mantener. La abstracción de componentes como campos de texto, etiquetas e íconos ayudó a evitar duplicación de código y facilitó realizar cambios globales en la interfaz sin tener que modificar cada pantalla individualmente. Además, el manejo de estados en Compose hace que la actualización de la UI sea mucho más dinámica y sencilla, permitiendo enfocarse más en la lógica y experiencia del usuario que en configuraciones repetitivas.

Sobre la integración de Firebase: Firebase me pareció una herramienta muy útil para complementar el proyecto, especialmente por la facilidad con la que se integra con aplicaciones Android desarrolladas en Compose. La sincronización de datos y el manejo en tiempo real ayudan a que la información mostrada en pantalla se actualice de forma inmediata, lo cual mejora la experiencia del usuario. También considero que Firebase simplifica mucho el trabajo del backend, ya que permite implementar autenticación, almacenamiento y persistencia de datos sin necesidad de configurar servidores complejos. Gracias a esto, fue posible dedicar más tiempo a mejorar el diseño y la estructura de la aplicación.